Estruturas em python

MÓDULO 1 SEMANA 03

@SCTI.SC

AGENDA

- Métodos em strings
- List Comprehensions
- Funções
- GitHub

2

O que são Métodos em Strings?

Strings em Python

Uma string é qualquer sequência de caracteres delimitada por aspas simples '...' ou duplas "...". Em Python, strings são imutáveis , ou seja, todo método cria uma nova string em vez de modificar a original.

Elas são sequências de caracteres : textos, nomes, frases e valores. Em Python, elas são um dos tipos de dados mais usados em análise de dados, automação e APIs.

Python Python

String Mamipulation

Stringnaxnfulation

String_String

```
wh
0
9       upper()
15      upper()
25      lower()
16      upper()
19      stiwer()
11      sowert()
34    }
25    {
18      upper()
20      lower()
43      lower()
30    {
15      split ()
20      split()
25      replit ()
18
10      replace()
23    }
11    {
        upper()
        cooris_strinn()_
      }
      }

        replace()
      }
        replace()
      }

      }
```

Strings são o tipo de dado mais comum em programação. Quase todo programa que lida com texto usa strings!

O que são métodos em Strings?

Um método é uma função que pertence a um objeto. Assim, métodos de strings são funções que já vêm prontas no tipo str e são usadas para manipular textos.

Em nosso cenário podemos dividir em:

- Métodos de Capitalização
- Métodos de Busca
- Métodos de Modificação
- Métodos de Divisão/Junção
- Métodos de Verificação
- Métodos de Formatação

Você não precisa importar nada!
Basta chamar com ponto (.)

1 2
variavel.metodo() 3

Objeto
Separador
Método
Parênteses

Métodos de Capitalização

Controlam como as letras aparecem, maiúsculas e minúsculas.

.upper()

Tudo em maiúsculas.

"python".upper()
→ "PYTHON"

.lower()

Tudo em minúsculas.

“python".lower()
→ “python"

.capitalize()

Primeira letra maiúsculas, o resto minúsculo.

“python É".capitalize()
→ “Python é"

.title()

Primeira letra de cada palavra maiúscula.

“python é incrível".title()
→ “Python É Incrível"

.swapcase()

Inverte a Caixa de cada caracter.

“Python".swapcase()
→ “pyTHON"

Dica: use .lower() ao comparar strings para evitar erros com maiúsculas/minúsculas.
Ex: if entrada.lower() == "sim":

Métodos de Busca

Localizam trechos e verificam a presença de substrings

.find()

Retorna o índice da 1ª ocorrências. Retorna -1 senão encontrar.

“Python".find(“y”)
→ 1

“Python".find(“z”)
→ -1

.index()

Igual ao .find(), mas lança ValueError senão encontrar.

“Python".index(“y”)
→ 1

“Python".index(“z”)
→ ValueError

.count()

Conta quantas vezes um trecho aparece na string.

“banana”.count(“a”)
→ 3

.startswith()

Retorna True se a string começam com o trecho.

“arquivo.pdf".starstwith(“arquivo”)
→ True

.endswith()

Retorna True se a string termina com o trecho.

“relatorio.ppt".endstwith(“ppt”)
→ False

Diferença chave: .find() é seguro (retorna -1). .index() lança erro , use com try/except ou quando tem certeza que o trecho existe.

Divisão e Junção

Convertem entre strings e listas, a operação é muita usada no dia a dia.

.split(sep)

Divide a string em uma lista usando sep como separador. Senão passar nada, divide por espaços.

```
"a,b,c".split(",")
→ ["a","b","c"]

"um dois três".split()
→ ["um","dois","três"]

"2024-01-15".split("-")
→ ["2024","01","15"]

"A:B:C".split(":",2)
→ ["A","B","C:..."]  (limite)
```

.join(lista)

Junta uma lista de strings usando a string como separador. Inverso do .split().

```
", ".join(["a","b","c"])
→ "a, b, c"

"-".join(["2024","01","15"])
→ "2024-01-15"
```

.partition(sep)

Divide em exatamente 3 partes: antes, separador, depois.

```
""Python é incrível".partition("é")
→ ("Python ", "é", " incrível")
```

Métodos de Verificação

Retornam TRUE ou FALSE são muito usados em condicionais e validações.

.isalpha()

Retorna se encontrar somente letras (sem números ou símbolos).

"Python".isalpha() → True
"Python3".isalpha() → False

.isdigit()

Retorna se encontrar somente dígitos numéricos.

"12345".isdigit() → True
"12.3".isdigit() → False

.isalnum()

Retorna se encontrar somente letras e números (sem espaços ou símbolos).

"Python3".isalnum() → True
"Python 3".isalnum() → False

.startswith()

Retorna se a string só tem espaços em branco.

" ".isspace() → True
" a ".isspace() → False

.islower()

Retorna se todas as letras estão em minúsculas.

"python".islower() → True
"Python".islower() → False

.isupper()

Retorna se todas as letras estão em maiúsculas.

“PYTHON".isupper() → True
"Python".isupper() → False

Formatação de Strings

Inserindo variáveis dentro de textos de forma limpa e legível.

.format()

Substitui {} na string pelos valores passados como argumentos.

"Olá, {}!".format("Ana")
→ "Olá, Ana!"

"{} tem {} anos".format("Ana", 25)
→ "Ana tem 25 anos"

"{nome}".format(nome="Ana")
→ "Ana" (por nome)

"{:.2f}".format(3.14159)
→ "3.14" (2 casas decimais)

.f-strings

Forma mais limpa e rápida, variáveis dentro do texto, prefixo f antes das aspas.

nome = "Ana"
f"Olá, {nome}!" → "Olá, Ana!"

idade = 25
f"Ela tem {idade} anos."

f"2 + 2 = {2+2}" → "2 + 2 = 4"

f"{3.14:.2f}" → "3.14"

Strings são Imutáveis

Nenhum método altera a string original. Eles sempre retornam uma string **NOVA.**
Se você não guardar o resultado em uma variável, ele se perde.

❌ Erro comum

```
texto = "python"

texto.upper()      # resultado perdido!

print(texto)       # → "python"


# O método foi chamado, mas

# a string não foi atualizada.
```

✓ Forma correta

```
texto = "python"

texto = texto.upper()  # guarda o resultado

print(texto)          # → "PYTHON"


# Ou use diretamente:

print(texto.upper())   # → "PYTHON"
```

Strings são Imutáveis

Cada método retorna uma string nova , então você pode chamar outro método logo em seguida, com um ponto. Isso é encadeamento (method chaining).

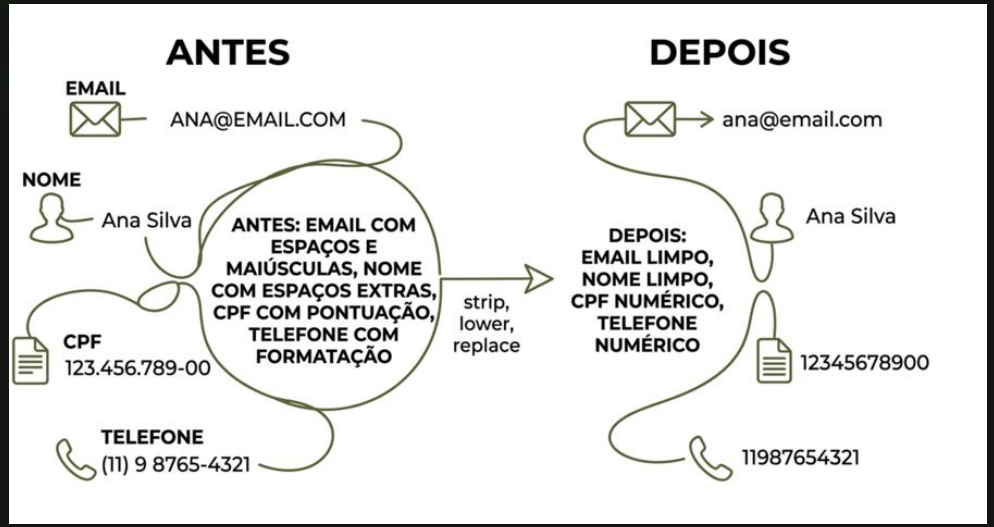
" python é incrível! " → .strip() → "python é incrível!" → .title() → "Python É Incrível!"

" python é incrível! ".strip().title() → "Python É Incrível!"

entrada.strip().lower()	→ remove espaços e coloca em minúsculas , ótimo para comparar entradas do usuário.
" ANA ".strip().lower().capitalize()	→ "Ana" — limpa, padroniza e capitaliza um nome.
"a, b, c".replace(" ","").split(",")	→ ["a","b","c"] — remove espaços e divide.
" ".join(frase.split())	→ remove espaços duplos de um texto.

Antes e Depois. Limpando dados reais!

Veja como podemos combinar vários métodos para transformar dados sujos em dados prontos para o uso:




```
email = "  ANA@EMAIL.COM  "
email_ok = email.strip().lower()   # ana@email.com

cpf = "123.456.789-00"
cpf_ok = cpf.replace(".", "").replace("-", "")  # 12345678900
```

Boas práticas com Strings

❌ O que evitar

Não ignore o retorno dos métodos (strings são imutáveis!)

Evite concatenar em loop com + — use .join()

Não compare strings sem normalizar a caixa

Não use .index() sem tratamento de erro

✓ O que Fazer

Sempre normalize com .strip() dados vindos de usuários
Use .lower() antes de comparar strings

Prefira f-strings para formatar texto

Salve o resultado dos métodos em variáveis

Use .find() quando a presença é incerta

Strings em ETL/ELT – No mundo real

Em pipelines de dados, os métodos de string são usados **o tempo todo** para limpar, transformar e padronizar informações antes de carregá-las em bancos de dados.

```
linha = "  MARIA SILVA , maria@email.com , 011-98765-4321
campos = [c.strip().lower() for c in linha.split(",")]
# ['maria silva', 'maria@email.com', '011-98765-4321']
```

Extract (Extrair)

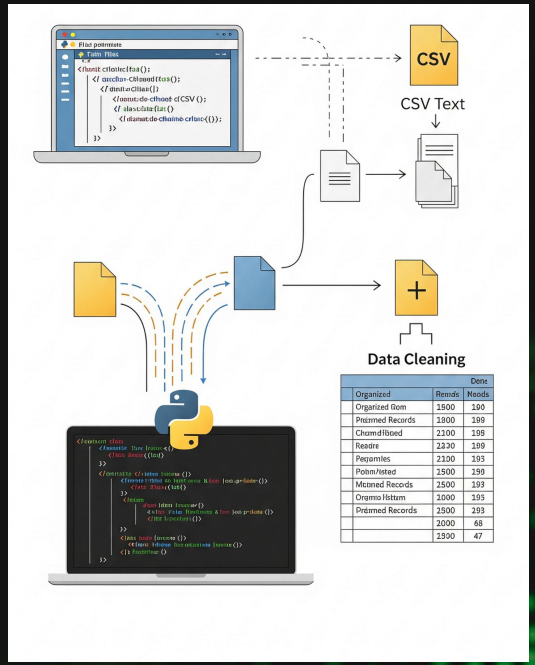
Dados chegam sujos: espaços extras, caixas mistas, pontuação indesejada em campos como CPF e telefone.

Transform (Transformar)

Aplicar .strip(), .lower(), .replace(), .split() para normalizar cada campo.

Load (Carregar)

Dados limpos e padronizados são carregados no banco de dados ou data warehouse sem erros de duplicidade.



Strings – Perguntas de Entrevistas

❓ Strings são mutáveis em Python?

❓ Como inverter uma string?

❓ Qual a diferença entre .find() e .index()?

❓ Como verificar se uma string contém outra?

Strings – Perguntas de Entrevistas

❓ Strings são mutáveis em Python?

Não. Strings são imutáveis. Todo método retorna uma nova string, sem modificar a original.

❓ Como inverter uma string?

Use slicing: texto[::-1].
Exemplo:
"python"[::-1] → "nohtyp".

❓ Qual a diferença entre .find() e .index()?

find() retorna -1 se não encontrar; .index() lança ValueError. Prefira .find() em situações incertas.

❓ Como verificar se uma string contém outra?

Use o operador in: "py" in "python" retorna True. Ou use .find() para obter a posição.

Strings – Mini Desafios!

Desafio 1

Receba um nome com espaços e retorne-o em maiúsculas e sem espaços nas bordas.

Dica: .strip().upper()

Desafio 2

Receba a string "João,30,Recife" e exiba cada campo separado em uma linha diferente.

Dica: .split(",") + loop

Desafio 3

Conte quantas vogais existem em uma frase digitada pelo usuário, ignorando maiúsculas e minúsculas.

Dica: .lower() + .count() para cada vogal

💡

Tente resolver sem ver a dica primeiro! O erro faz parte do aprendizado em programação.

Vamos nos preparar?

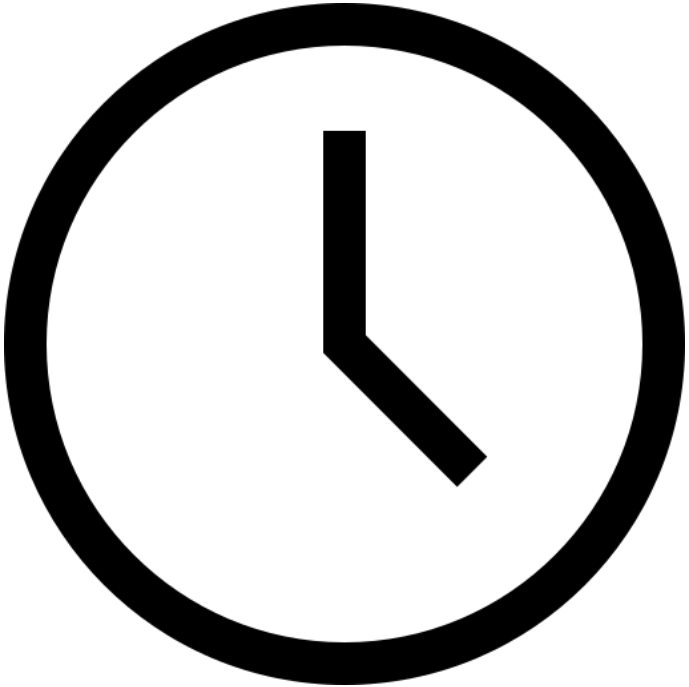
Vamos resolver usando métodos em strings.

INTERVALO!

Finalizamos o nosso primeiro período de hoje. Que tal descansar um pouco?!

Nos vemos em 15 minutos.

Início: 20:30
Retorno: 20:45



O que são List Comprehensions?

O que é List Comprenhesion?

List Comprehension é uma forma concisa e elegante de criar listas em Python. Em vez de usar um loop for tradicional com várias linhas, você expressa a criação de uma lista em uma única linha de código.

Essa sintaxe é inspirada na notação matemática de conjuntos , por exemplo, "o conjunto de todos os x^2 para x no intervalo de 1 a 10". Python traduz essa ideia diretamente para código.

List Comprehension não é apenas um atalho , ela comunica **intenção** de forma muito mais clara do que loops equivalentes.

Notação matemática vs. Python

Matemática: $\{ x^2 \mid x \in \{1,2,3,4,5\} \}$

```
[x**2 for x in range(1, 6)]

# Resultado: [1, 4, 9, 16, 25
```

Simples assim! A List Comprehension lê-se quase como uma frase em inglês: *"x ao quadrado, para cada x no intervalo de 1 a 6"*.

E a sua sintaxe?

É uma sintaxe especial do Python que permite criar listas em uma única linha de forma legível e eficiente. É considerada umas das features mais compactas de criar listas.

Anatomia da sintaxe:

```
[ expressao for variavel in iteravel if condicao ]
```

- ⬆️ O que você quer na lista
- ⬆️ Palavra-chave do Python
- ⬆️ Nome para cada item
- ⬆️ Palavra-chave do Python
- ⬆️ Lista, range ou coleção
- ⬆️ Opcional: filtro

O Problema: Criar Listas com For Loop

Imagine que você quer criar uma lista com os quadrados de 1 a 5:

```
# Jeito tradicional: for loop
quadrados = []
for numero in range(1, 6):
    quadrados.append(numero ** 2)

print(quadrados)  # [1, 4, 9, 16, 25
```

Funciona... mas:

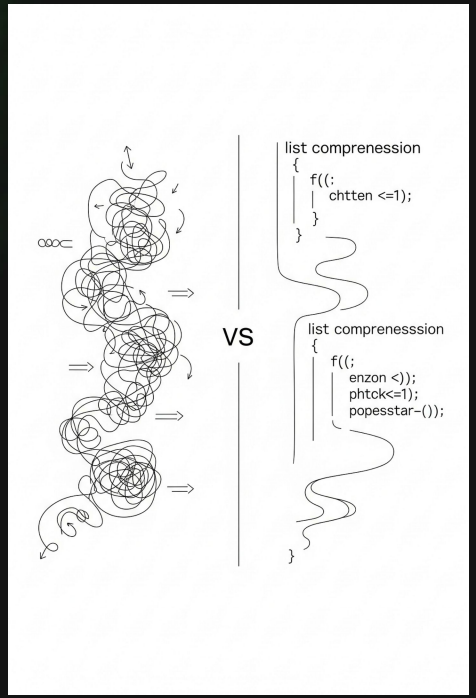
- ❌ 4 linhas para uma tarefa simples
- ❌ Variável criada antes de ser usada
- ❌ Menos legível para o leitor
- ❌ Mais lento de escrever

💡 **Existe um jeito mais elegante...**

```
# Usando List Comprehension ✨
quadrados = [numero ** 2 for numero in range(1, 6)]

print(quadrados)  # [1, 4, 9, 16, 25]
```

Porque usar List Comprehension?



🚀 Código Mais Rápido

List Comprehensions são otimizadas internamente pelo Python e geralmente **30–50% mais rápidas** do que loops equivalentes com append().

📖 Mais Legível

Uma única linha bem escrita comunica a intenção com clareza, tornando o código mais fácil de ler e manter por outros desenvolvedores.

✂️ Menos Código

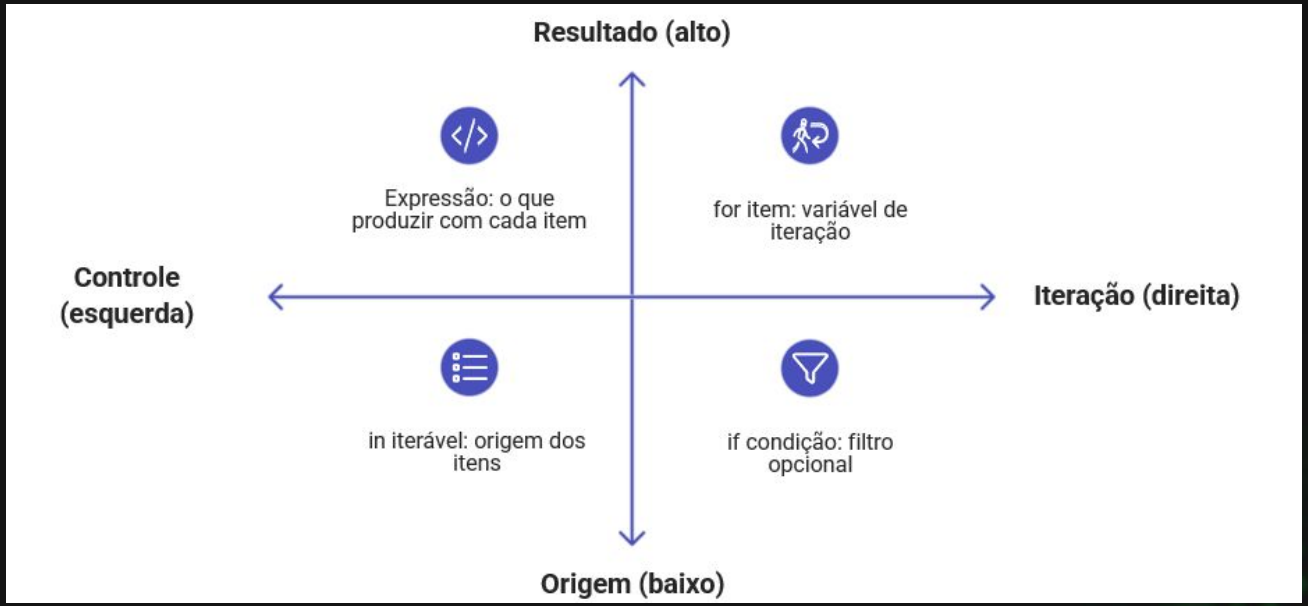
Reduz de 4–5 linhas de loop para **uma única expressão**, diminuindo a chance de erros e tornando o código mais conciso.

🐍 Pythônico

É considerado o estilo **"Pythônico"** , a forma idiomática recomendada pela comunidade Python para transformar e filtrar listas.

Forma Básica x Forma Completa

Entenda cada parte da estrutura de uma List Comprehension antes de sair usando.



Exemplos

1. Lista de números

```
# Números de 0 a 9
numeros = [x for x in range(10)]
# [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

2. Dobrar valores

```
# Dobrar cada número
dobros = [x * 2 for x in range(5)]
# [0, 2, 4, 6, 8]
```

3. Quadrados

```
# Quadrados de 1 a 5
quad = [x**2 for x in range(1,6)]
# [1, 4, 9, 16, 25]
```

4. Letras de uma palavra

```
# Extrair letras de uma string
letras = [c for c in 'Python']
# ['P','y','t','h','o','n']
```

5. Converter strings para maiúsculo

```
frutas = ['maçã','banana','uva']
maiusc = [f.upper() for f in frutas]
# ['MAÇÃ', 'BANANA', 'UVA']
```

6. Comprimento de palavras

```
palavras = ['gato','elefante','pato']
tamanhos = [len(p) for p in palavras]
# [4, 8, 4]
```

Forma Básica x Forma Completa

Forma Básica (sem filtro)

```
[expressão for item in iterável]

# Exemplo:
nomes = ["ana", "bob", "cia"]
maiusculas = [n.upper() for n in nomes]
# ['ANA', 'BOB', 'CIA']
```

Forma Completa (com filtro)

```
[expressão for item in iterável if condição]

# Exemplo:
nums = [1, 2, 3, 4, 5, 6]
pares = [n for n in nums if n % 2 == 0]
# [2, 4, 6]
```

Podemos usar o IF/ELSE x no Filtro

São usos diferentes. Cuidado para não confundir!

if no FILTRO (ao final)

Inclui ou exclui elementos

```
numeros = [1, 2, 3, 4, 5]

# Só inclui os pares
pares = [
  x
  for x in numeros
  if x % 2 == 0
]

# Resultado: [2, 4]
```

if/else na EXPRESSÃO (no início)

Transforma o valor, não filtra

```
numeros = [1, 2, 3, 4, 5]

# Transforma cada número
result = [
  'par' if x % 2 == 0
  else 'ímpar'
  for x in numeros
]

# ['ímpar','par','ímpar','par','ímpar']
```

Regra prática: if no final = filtro (pode ter menos itens). if/else no início = transformação (sempre o mesmo número de itens).

For Loop x List Comprehension

1. Criar lista de quadrados

For loop

```
quadrados = []
for x in range(1, 6):
    quadrados.append(x**2)
```

List Comprehension

```
quadrados = [x**2 for x in range(1, 6)]
```

2. Maiúsculas de nomes

For loop

```
maiusc = []
for nome in nomes:
    maiusc.append(nome.upper())
```

List Comprehension

```
maiusc = [nome.upper() for nome in nomes]
```

3. Filtrar números pares

For loop

```
quadrados = []
for x in range(1, 6):
    quadrados.append(x**2)
```

List Comprehension

```
pares = [x for x in range(20) if x % 2 == 0]
```

Casos de uso reais

Onde podemos usar as lists comprehensions no dia a dia.

```
# 1. Limpar dados de uma planilha
nomes_brutos = ['  Ana ', 'CARLOS  ', ' maria']
nomes_limpos = [n.strip().title() for n in nomes_brutos]
# ['Ana', 'Carlos', 'Maria']

# 2. Extrair emails de uma lista de usuários
usuarios = [{'nome': 'Ana', 'email': 'ana@gmail.com'},
            {'nome': 'João', 'email': 'joao@yahoo.com'}]
emails = [u['email'] for u in usuarios]
# ['ana@gmail.com', 'joao@yahoo.com']

# 3. Converter temperaturas de Celsius para Fahrenheit
celsius = [0, 20, 37, 100]
fahrenheit = [(c * 9/5) + 32 for c in celsius]
# [32.0, 68.0, 98.6, 212.0]

# 4. Remover duplicatas mantendo ordem
com_repeticao = [1, 2, 2, 3, 1, 4, 3]
sem_rep = list({x for x in com_repeticao})  # set comprehension!
# [1, 2, 3, 4]
```

Boas práticas

Mantenha simples e legível

```
# ❌ Complexo demais — use for loop!
result = [f(x) for x in range(100) if g(x) > 0 if h(x) < 10]
```

```
# ✅ Simples e claro
quad = [x**2 for x in range(10)]
```

Use nomes descritivos

```
# ❌ Confuso
x = [a * 0.9 for a in b
```

```
# ✅ Nome descritivo
precos_com_desc = [p * 0.9 for p in precos]
```

Prefira for loop para lógica complexa

```
# ❌ Difícil de ler
r=[transformar(calcular(x)) for x in dados if validar(calcular(x))]
```

```
# ✅ Use for quando tiver muita lógica
result = []
for x in dados:
    y = calcular(x)
    if validar(y):
        result.append(transformar(y))
```

Perguntas de Entrevista

Q1: O que é uma list comprehension e qual a vantagem sobre um for loop?

Q2: Qual a diferença entre 'if no final' e 'if/else no início'?

Q3: Quando NÃO usar list comprehension?

Q4: Quando usar list comprehension e quando evitar?

Perguntas de Entrevista

Q1: O que é uma list comprehension e qual a vantagem sobre um for loop?

É uma sintaxe compacta para criar listas. Vantagens: código mais conciso, mais legível para operações simples, e ligeiramente mais rápido que o for loop equivalente por ser otimizado internamente.

Q2: Qual a diferença entre 'if no final' e 'if/else no início'?

if no final filtra elementos (a lista resultado pode ter menos itens). if/else no início transforma elementos (a lista resultado tem sempre o mesmo número de itens).

Q3: Quando NÃO usar list comprehension?

Quando a lógica for complexa (múltiplos ifs, funções complexas) — prefira for loop para manter legibilidade. Também evite para grandes volumes de dados onde generator expressions são mais eficientes em memória.

Q4: Quando usar list comprehension e quando evitar?

List comprehension use quando a lógica for simples e direta, deixando o código mais curto e legível. Evite quando a expressão ficar complexa ou difícil de entender, nesses casos podemos usar um for tradicional.

Vamos nos preparar?

Vamos resolver usando
List Comprehesion!

Parte 1

01 Crie uma lista com os números de 1 a 20.

💡 *Dica: Use range(1, 21)*

02 A partir da lista [3, 7, 1, 9, 4, 6, 2, 8, 5], crie uma lista apenas com os valores maiores que 5.

💡 *Dica: Use condição if*

03 Dada a lista de nomes ['ana', 'pedro', 'maria', 'jose'], crie uma nova lista com os nomes em letras maiúsculas.

💡 *Dica: .upper()*

04 Crie uma lista com os múltiplos de 3 entre 1 e 30.

💡 *Dica: Use x % 3 == 0*

05 Dada lista de preços [100, 250, 80, 320, 150], aplique 10% de desconto em todos.

💡 *Dica: p * 0.9*

Parte 2

06

Dada a string 'Python é incrível', crie uma lista com cada palavra que tem mais de 4 letras.

💡 *Dica: s.split() para separar palavras*

07

Crie uma lista que substitui números negativos por 0, mantendo os positivos. Lista: [-3, 5, -1, 8, -7, 2].

💡 *Dica: Use if/else na expressão (no início)*

08

Dada a lista de strings ['123', 'abc', '456', 'def', '789'], crie uma lista apenas com os valores que são números (use .isdigit()).

💡 *Dica: .isdigit() retorna True/False*

09

A partir de uma lista de temperaturas em Celsius [0, 15, 22, 30, 38], converta para Fahrenheit apenas as temperaturas acima de 20°C.

💡 *Dica: Combine filtro e transformação*

10

Crie uma lista de tuplas (número, quadrado, cubo) para os números de 1 a 5.

💡 *Dica: A expressão pode ser uma tupla: (x, x**2, x**3)*

Estruturas em python

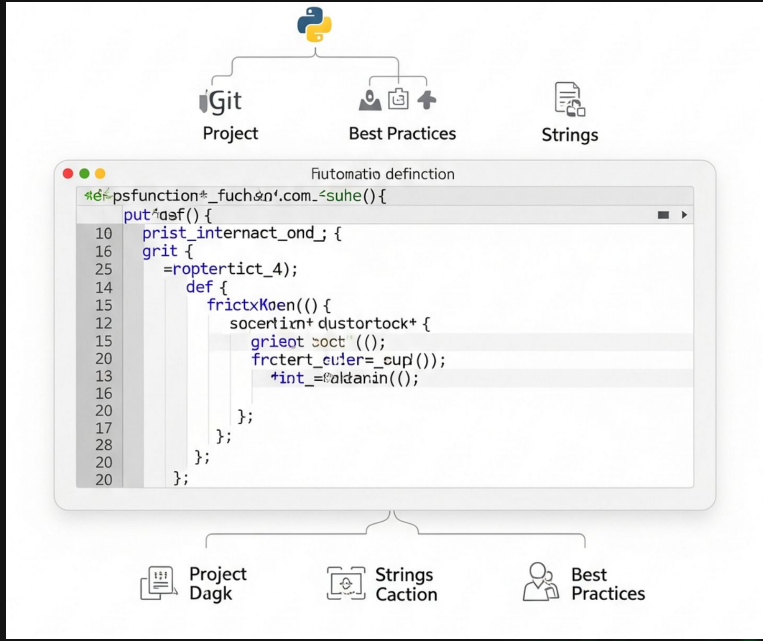
Aula 2

@SCTI.SC

O que são Funções?

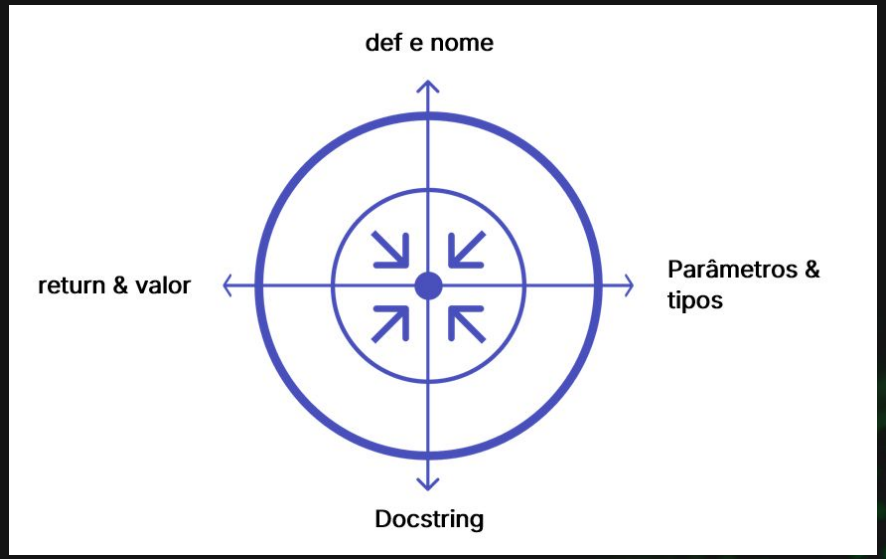
Funções em Python

Funções são blocos de código reutilizáveis que realizam tarefas específicas. Elas tornam o código mais legível, testável e fácil de manter, e é um dos princípios fundamental do desenvolvimento profissional.



Anatomia de uma Função em Python

Uma função bem escrita tem nome claro, parâmetros com tipos indicados, documentação (docstring) e um retorno explícito. Esse padrão é esperado em ambientes profissionais e em entrevistas técnicas.



Dica de entrevista: Uma função bem escrita tem nome claro, parâmetros com tipos indicados, documentação (docstring) e um retorno explícito. Esse padrão é esperado em ambientes profissionais e em entrevistas técnicas.

Estrutura de uma função

Uma função é um bloco de código com nome próprio que executa uma tarefa. Você a define uma vez e pode chamá-la quantas vezes quiser , de qualquer lugar.

Benefícios:

- ✓ Evita repetição de código
- ✓ Facilita manutenção
- ✓ Divide problemas complexos
- ✓ Melhora a legibilidade

Estrutura de uma função:

```
def   nome_da_funcao   ( param1, param2 ):
    """Descrição da função"""
    # corpo da função
    return resultado
```

```
def somar(a, b):
    """Soma dois números"""
    return a + b

resultado = somar(3, 5)
print(resultado)  # → 8
```

Parâmetros e Argumentos

Como passar dados para dentro de uma função. Podemos usar 5 formas diferentes.

Parâmetros simples

Definidos na criação da função e passados na ordem.

```
def somar(a, b):
    return a + b
somar(3, 5)  # → 8
```

Valor padrão (default)

Parâmetro com valor pré-definido. Opcional.

```
def saudacao(nome, msg='Olá'):
    return f'{msg}, {nome}!'
saudacao('Ana')    # → 'Olá, Ana!'
saudacao('Ana','Oi') # → 'Oi, Ana!'
```

Argumentos nomeados (keyword)

Passa pelo nome do parâmetro. Ignora a ordem.

```
def pessoa(nome, idade, cidade):
    return f'{nome}, {idade}'
pessoa(idade=25, nome='Ana',
       cidade='SP')  # ✓
```

Parâmetros e Argumentos

Como passar dados para dentro de uma função. Podemos usar 5 formas diferentes.

*args

Recebe qualquer número de argumentos posicionais como tupla.

```
def somar(*nums):
    return sum(nums)
somar(1, 2, 3, 4)  # → 10
somar(10, 20)      # → 30
```

**kwargs – Pares chave-valor

Recebe qualquer número de argumentos nomeados como dict

```
def exibir(**dados):
    for k,v in dados.items():
        print(f'{k}: {v}')
exibir(nome='Ana', idade=25)
```

AVALIAÇÃO DOCENTE

O que você está achando das minhas aulas neste conteúdo?

Clique aqui ou escaneie o QRCode ao lado para avaliar minha aula.

Sinta-se à vontade para fornecer uma avaliação sempre que achar necessário.

SCTEC

PREPARANDO
VOCÊ PARA O
AGORA_

@SCTI.SC